

DFD

Data Flow Diagram

DFD(Data Flow Diagram)

- **Developing Data Flow Diagrams(DFD)**
 - **a) What are DFDs?**
 - **b) Symbols used in DFD**
 - **c) Rules of data flow**
 - **d) Good style in drawing DFD**
- **Describing systems with DFD & Levelling DFDs**
- **Logical & Physical DFDs**

- A Data Flow Diagram (DFD) is a graphical tool used to represent how data moves through a system. It shows data inputs, outputs, data stores, and the processes that transform the data. DFDs provide a high-level overview of system functionality and are widely used in structured analysis due to their simplicity and clarity for both technical and non-technical users.

WHY DFD ?

Provides an overview of

- What data a system processes
- What transformations are performed
- What data are stored
- What results are produced and where they flow

WHY DFD ?

- Graphical nature makes it a good communication tool between
 - User and analyst
 - Analyst and System designer
- Structure of DFD allows starting from a broad overview and expand it to a hierarchy of detailed diagrams

WHAT ARE DATA FLOW DIAGRAMS (DFD's)

DFDs models the system by depicting

- External entities from which the data flows and where results terminate
- Processes which transform data flows
- Data stores from which the data are read or into which data are written by the processes.

DFD components

- **Graphical Representation**

DFD is a *diagrammatic (visual)* representation of how data moves through a system.

- **Focus on Data Flow**

It shows how data enters, moves within, and leaves the system.

- **Process-Oriented**

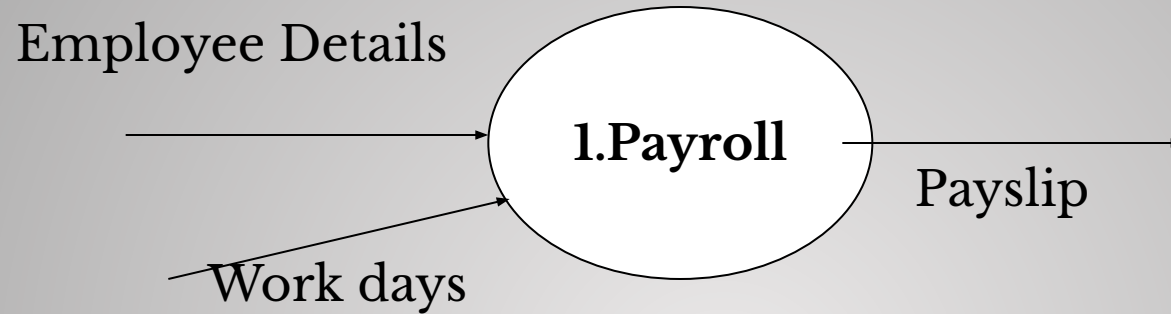
Emphasizes processes that transform input data into output data.

- **Logical Model**

Describes *what the system does*, not *how it is implemented* (technology-independent).

SYMBOLS USED IN DFD

PROCESS



- A circle represents a process
- Straight lines with incoming arrows are input data flows
- Straight lines with outgoing arrows are output data flows
- Processes are given serial numbers for easy reference
- Labels are assigned to Data flow. These aid documentation

SYMBOLS USED IN DFD

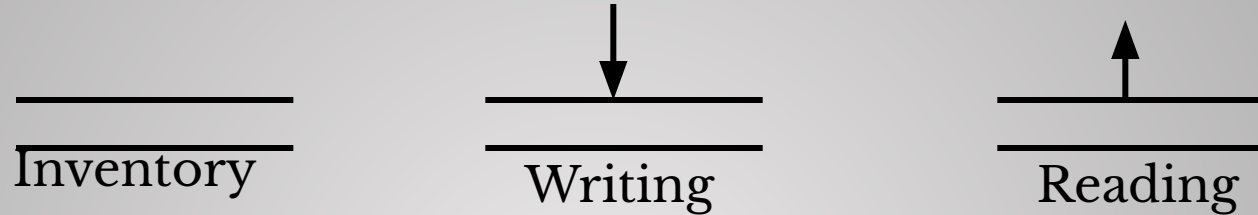
EXTERNAL ENTITIES



- A **Rectangle** represents an external entity
- They either supply data or receive data
- They **do not process data**

SYMBOLS USED IN DFD

DATA STORES



- A Data Store is a repository of data
- Data can be **written into the data store**

This is depicted by an incoming arrow

- Data can be **read from a data store**

This is depicted by an outgoing arrow

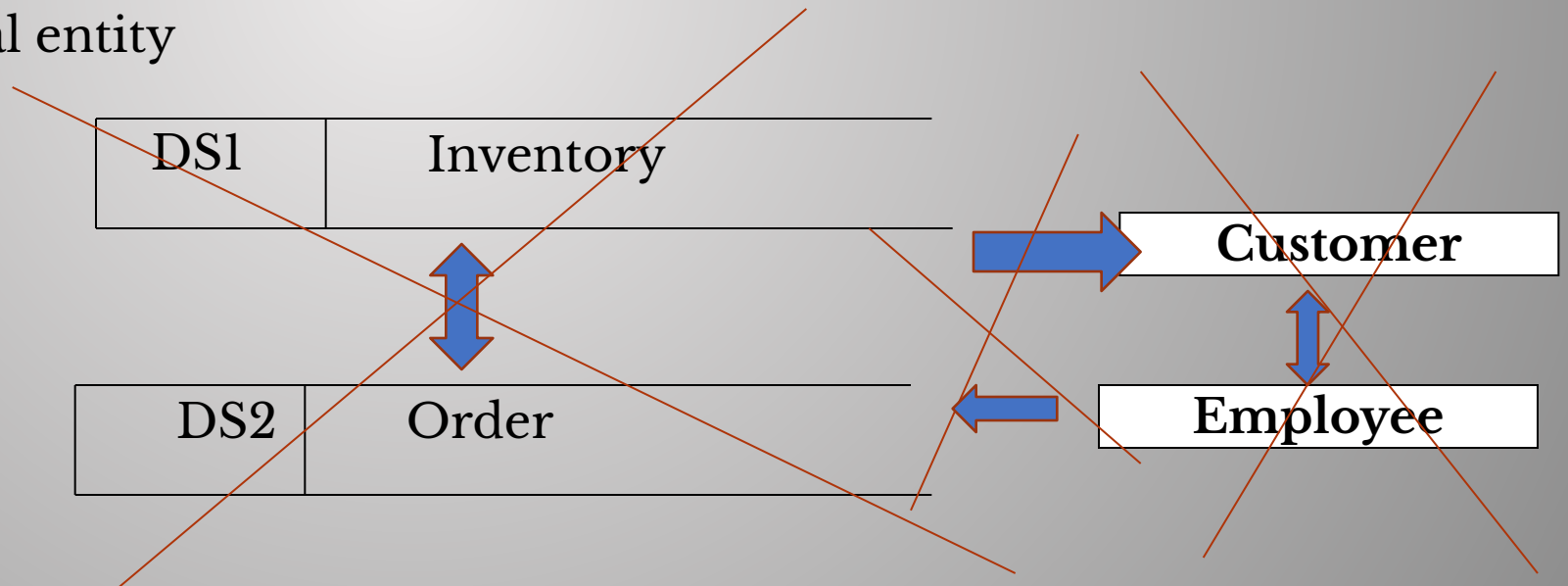
- **External entity** cannot read or write to the data store
- **Two data stores** cannot be connected by a data flow

RULES OF DATA FLOW

- Data can flow from
 - external entity to process
 - process to external entity
 - process to store and back
 - process to process
- Data cannot flow from
 - external entity to external entity
 - external entity to store
 - store to external entity
 - store to store

A Data store

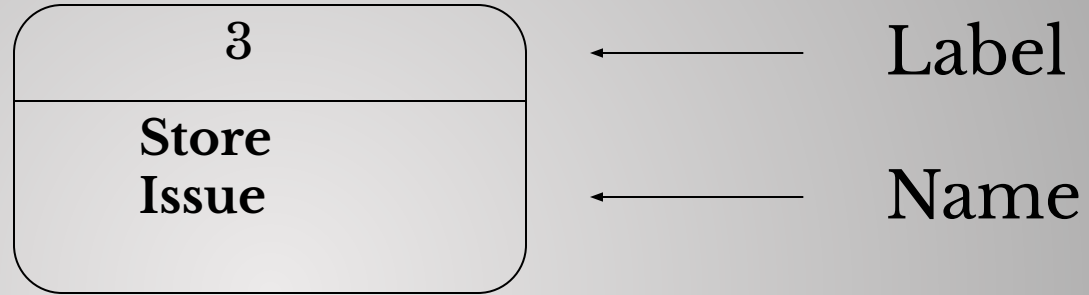
A Data store



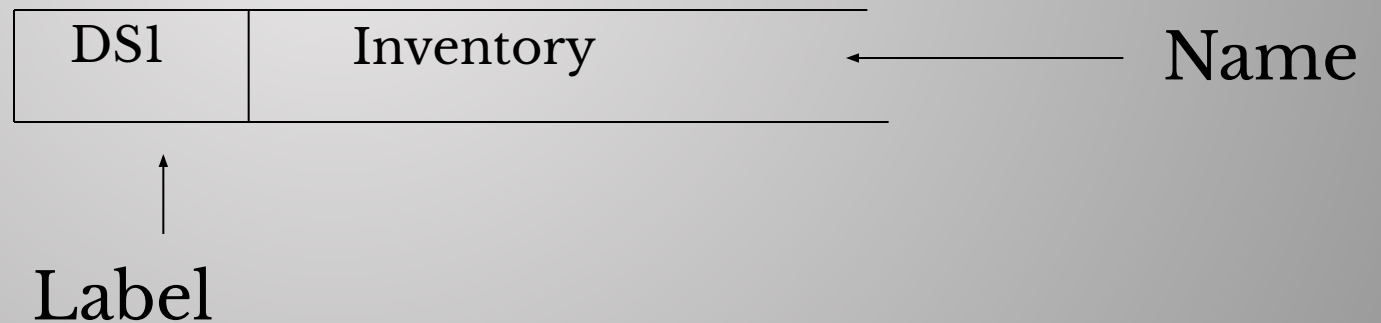
DATA FLOW DIAGRAMS

An alternate notation is often used

A Process



A Data store



GOOD STYLE IN DRAWING DFD

- Use **meaningful names** for data flows, processes and data stores.
- Use **top down development** starting from **context diagram** and successively levelling DFD
- Only **previously stored data can be read**
- A process can **only transfer input to output** .It cannot create new data
- **Data stores** cannot create new data

Creating Data Flow Diagrams

Steps:

1. Create a **list of activities**
2. Construct **Context Level DFD**
(identifies external entities and processes)
3. **Construct Level 0 DFD**
(identifies manageable sub process)
4. **Construct Level 1- n DFD**
(identifies actual data flows and data stores)
5. **Check against rules of DFD**

DFD Naming Guidelines

- External Entity ☐ Noun
- Data Flow ☐ Names of data
- Process ☐ verb phrase
 - a system name
 - a subsystem name
- Data Store ☐ Noun

LEVELLING DFD

- A context diagram **gives an overview**
- It should be **split into major processes** which give **greater detail**.
- Each **major process** is further **split to give more detail**.
- Each major process is further split to give more detail

WHY LEVEL DFD?

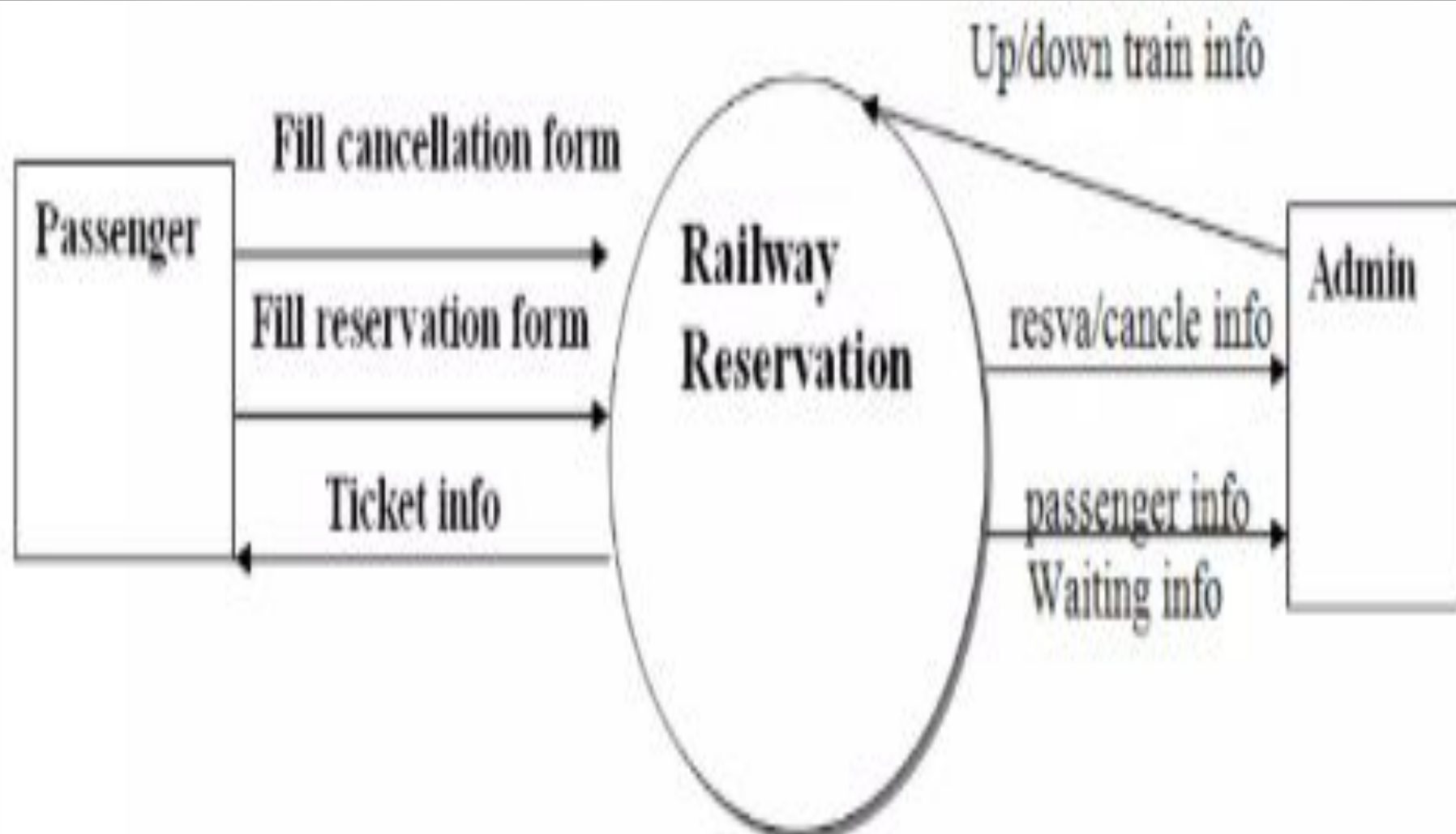
- If a DFD is **too detailed** it will have **too many data flows** and will be large and difficult to understand
- **Start from a broad overview.** Expand to details - Idea similar to using procedures and linking these with a main program
- Each DFD must deal with **one aspect of a big system**

Levels of DFD

- 0- Level
- 1- Level
- 2- Level

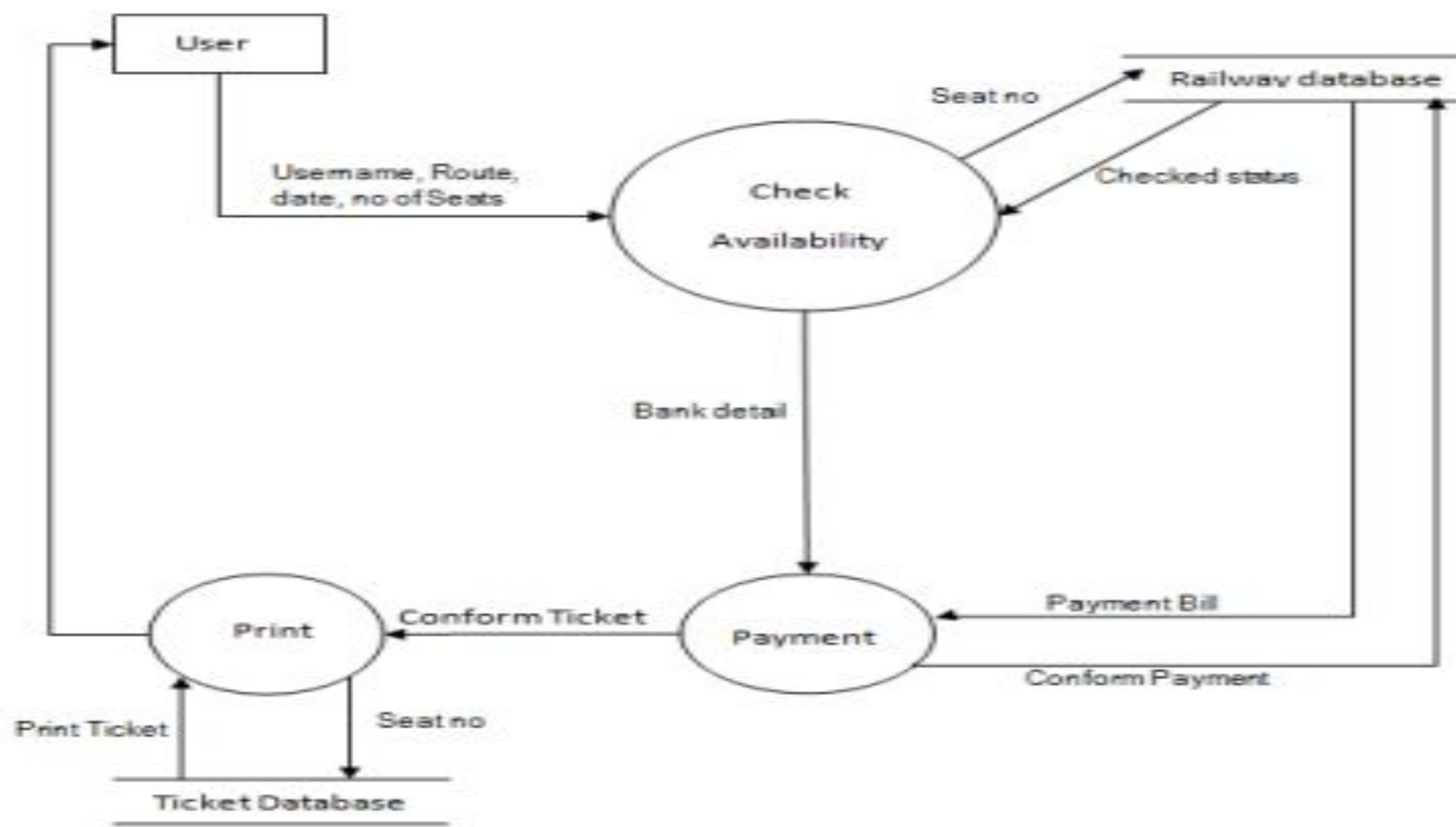
0 — Level DFD

- An abstraction view.
- This is also known as a context diagram.
- The full system is shown as a single process with its relationship to external entities.
- System as a single bubble with input and output data indicated by incoming/outgoing arrows.
- Meaningful label of the line is used to make more informative.
- This layers shows only the external outlook of the system. No need to show data store in this level.

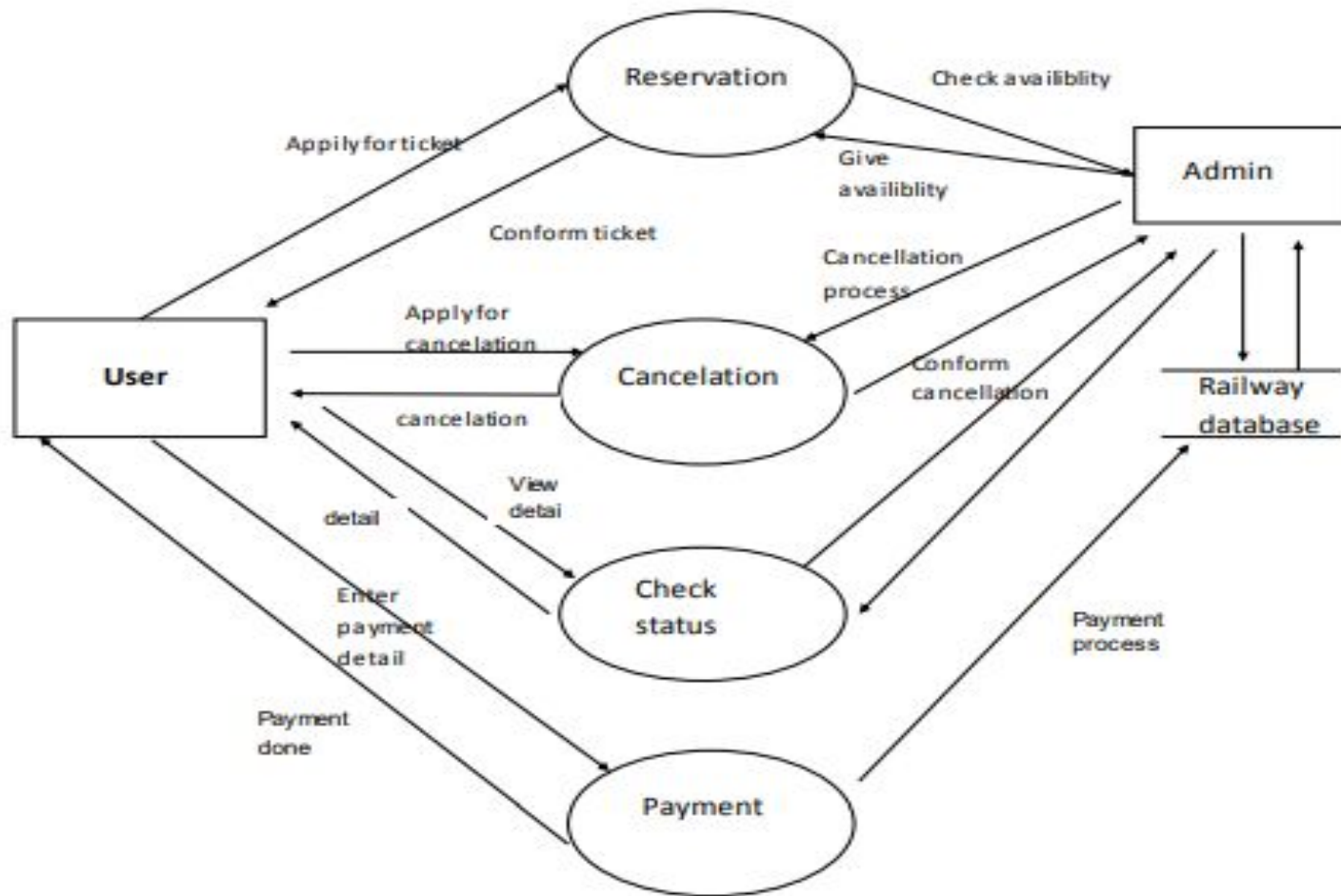


1- Level DFD

- Level 0 context diagram is decomposed into multiple bubbles/processes.
- Main functions of the system is highlighted.
- The high-level process of 0-level DFD broken-down into sub-processes here.
- It provides a more detailed view of the Context Level Diagram or **Level 0**
- Need to use the same entities as Level 0.



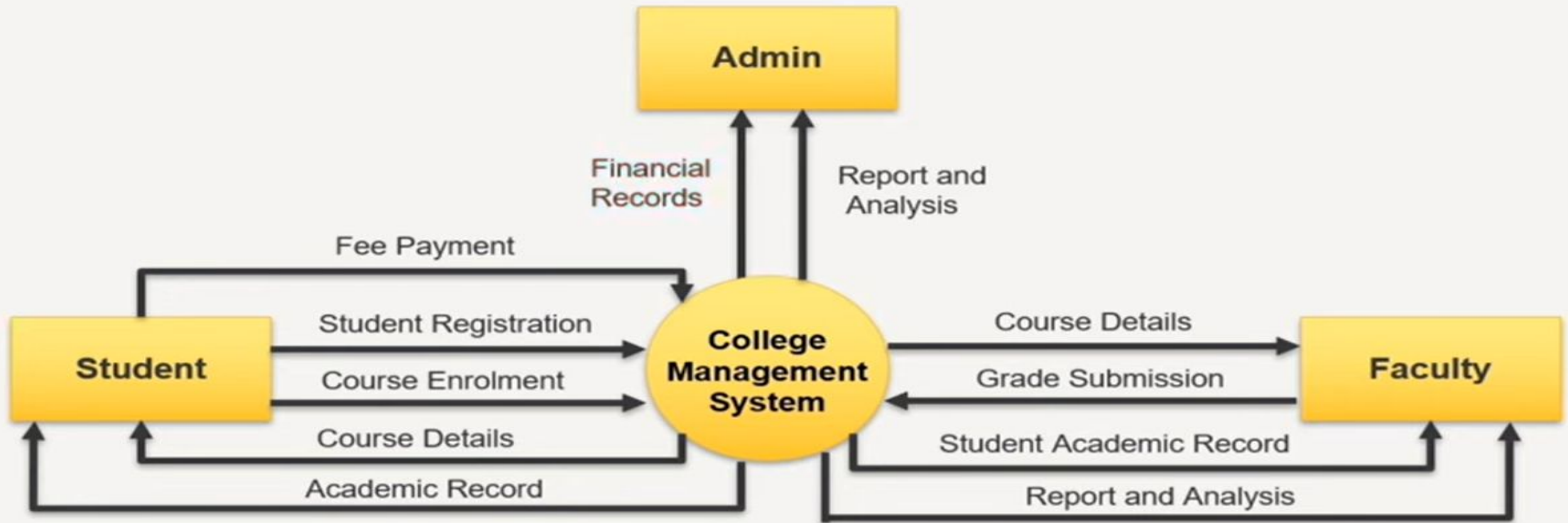
2- Level DFD



Level 0 DFD -Context Diagram

College Management System – Level 0

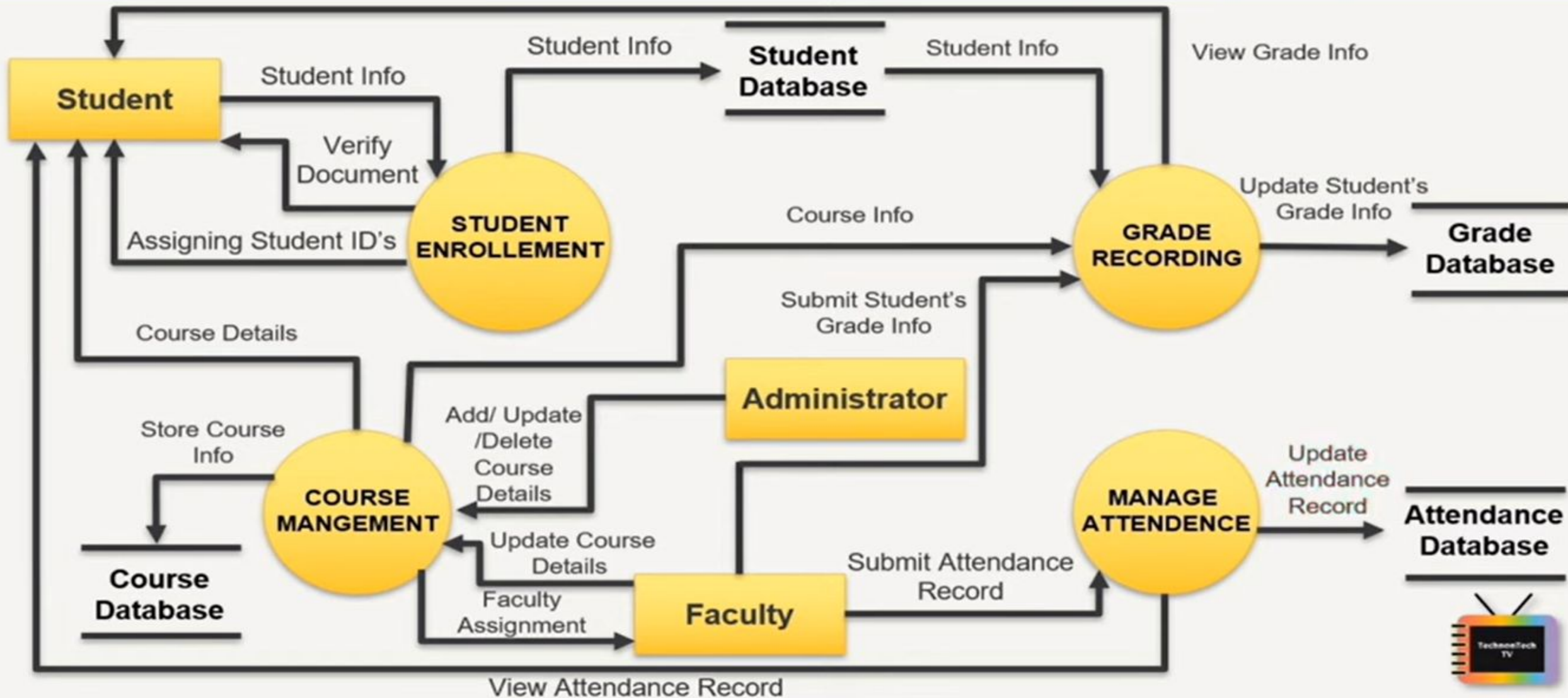
DFD Diagram



Level 1 DFD

College Management System – Level 1

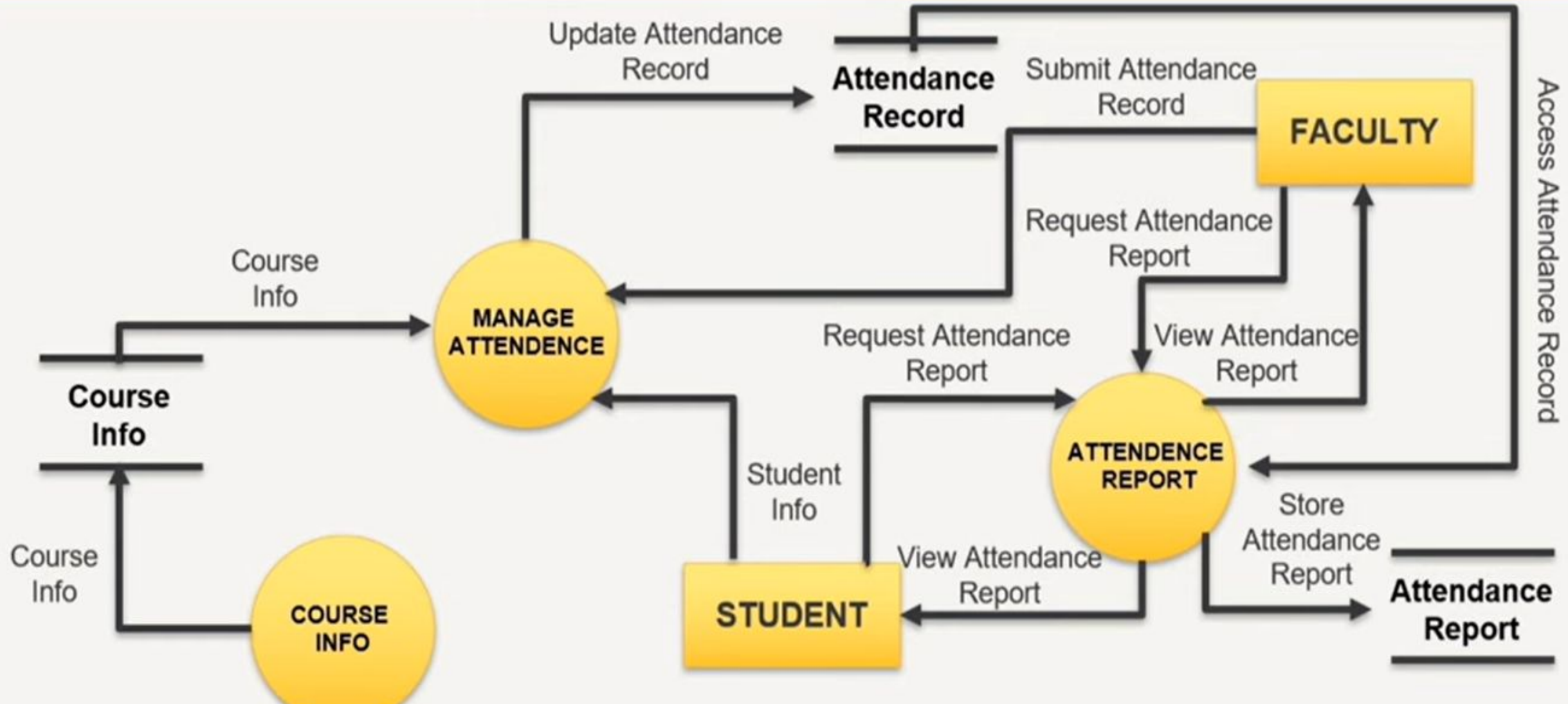
DFD Diagram



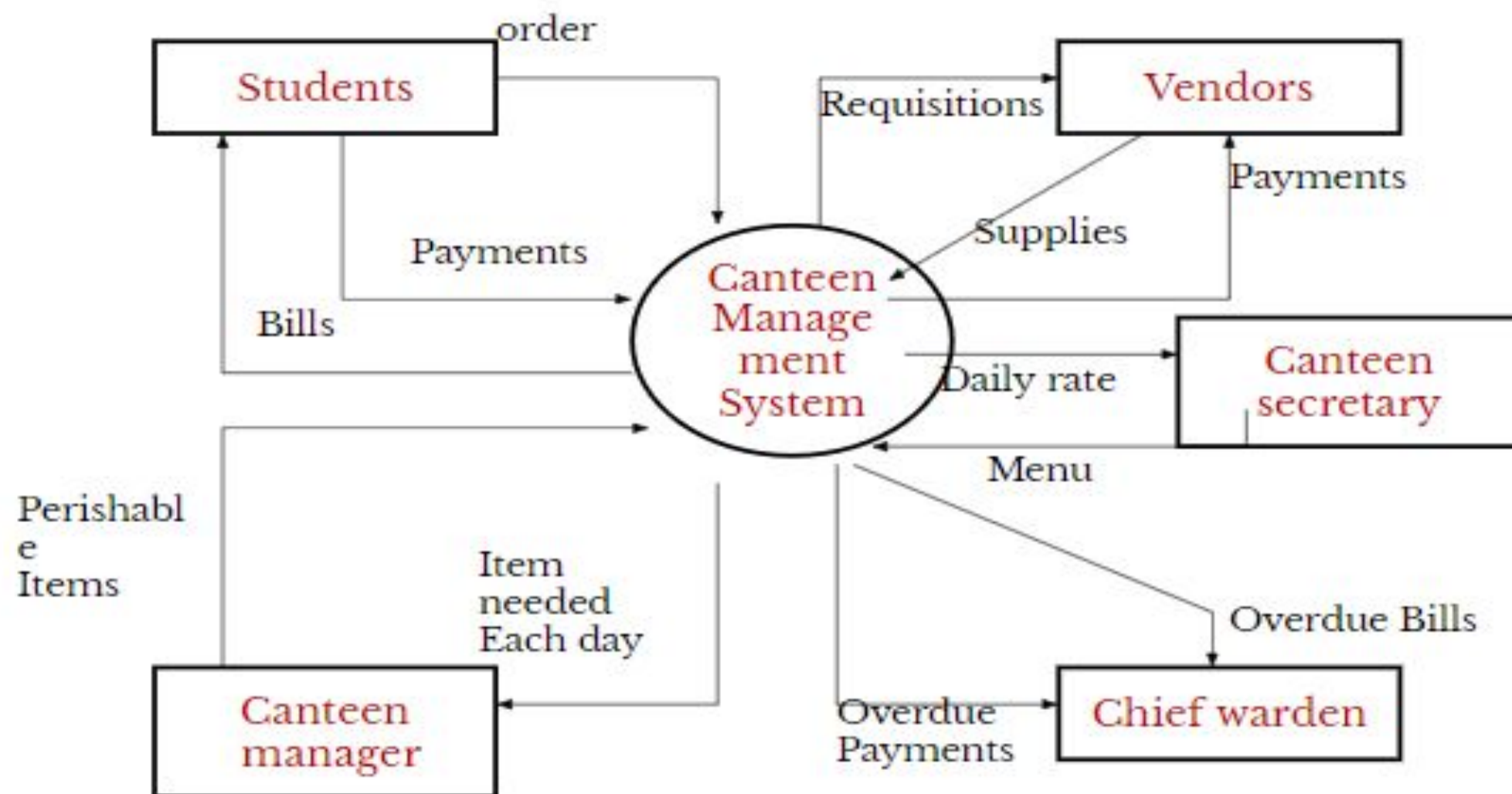
Level 2 DFD

College Management System – Level 2

DFD Diagram



CONTEXT DIAGRAM OF MESS MANAGEMENT SYSTEM



EXPANDED DFD FOR CANTEEN MANAGEMENT SYSTEM

